Artifact Submission Guide for Reproducibility

Material for "Variational Inference for De Finetti Logic"

Sami Hadouaj*, Ouael Ben Amara†, and Niccolò Meneghetti‡

University of Michigan-Dearborn

November 13, 2026

1 Introduction

This document provides a step-by-step guide for reproducing the experiments presented in our paper [2], "Variational Inference for De Finetti Logic". The paper extends StarfishDB [1], a query execution engine for relational probabilistic programming whose original inference engine is based on Collapsed Gibbs Sampling (CGS), with a Stochastic Variational Inference (SVI) engine.

The experiments reproduce **Table 1** of the paper: the test log-likelihood of Latent Dirichlet Allocation (LDA) trained with four approaches, evaluated at two timestamps, T1 and T2, on two datasets (PubMed with 100 topics and Wikipedia with 200 topics). All experiments use 24 threads. The four approaches are:

- **SFDB-SVI**: StarfishDB with the SVI engine presented in the paper;
- **SFDB-CGS**: StarfishDB with its CGS engine;
- **C++SVI**: a standalone C++ implementation of SVI for LDA;
- **Mallet**: the Mallet toolkit [4].

Reproducing these experiments is challenging because of the size of the datasets and because StarfishDB relies on just-in-time compilation (ClangJIT), which requires a customized toolchain. To make the process reliable, everything runs inside a Docker container, and a single script (`main_script.sh`) downloads the dependencies and the datasets, builds all the tools, runs all the experiments, and produces Table 1. The whole process takes about 4 days on our lab server. Internally, the process is divided into several scripts, each with its own log file, so that any failure can be identified and the corresponding step can be re-run on its own.

We appreciate the reviewers' efforts in verifying our work. If you encounter any difficulties during the reproduction process, please contact the authors; we are committed to supporting you. Thank you for your time and contribution to the robustness of our research.

*shadouaj@umich.edu
†benamara@umich.edu
‡niccolom@umich.edu

2 Overview

The reproduction consists of the following steps:

1. **Setting up the machine** (Section 3): install Docker, Git and tmux on the host and clone the repository.
2. **Creating the container** (Section 4): build the Docker image and create the container. The repository is mounted inside the container, so all outputs are also visible from the host.
3. **Running everything** (Section 4): run `main_script.sh` inside the container. Section 5 details each of its steps.
4. **Checking the results** (Section 6): compare the reproduced Table 1 with the one in the paper.

Outputs

- `benchmarks/csv/`: the results of each run, one set of CSV files per approach and dataset:
  - `*_loglik.csv`: log-likelihood at each checkpoint;
  - `*_plotdata.csv`: cumulative training time and perplexity at each checkpoint;
  - `*_perplexity.csv`: perplexity at each checkpoint;
  - `*_time.csv`: training time of each iteration and timing run.
- `logs/`: the output of each step, saved to `logs/log_<script name>.txt`.
- Terminal and `logs/log_make_loglik_table.txt`: the reproduced Table 1 and its comparison with the values reported in the paper.
- `report/loglik_tables.pdf`: the same two tables, typeset.

The names of the `loglik` and `plotdata` files, which are used to compute Table 1, follow these conventions, where `[DATASET]` is `PUBMED` or `wiki08`, `[SPLIT]` is `test` or `train`, and `[METRIC]` is `loglik` or `plotdata`. The `perplexity` and `time` files have similar names.

- SFDB-SVI:

  `[DATASET]_[SPLIT]_[TOPICS]topics_batchS[BATCH]_totalNumIter[ITERATIONS]_NT[THREADS]_RND[SEED]_gammapdb_VI_ViIterBatch[VI_ITER]_[METRIC].csv`

- SFDB-CGS:

  `[DATASET]_[SPLIT]_[TOPICS]topics_A[ALPHA]_B[BETA]_NI[ITERATIONS]_NT[THREADS]_RND[SEED]_gammapdb_lda-inmemory-vrexprP_[METRIC].csv`

- C++SVI:

  `[DATASET]_[SPLIT]_[TOPICS]topics_batchS[BATCH]_totalNumIter[ITERATIONS]_NT[THREADS]_RND[SEED]_cxx_SVI_lda_vi_iter_per_batch[VI_ITER]_[METRIC].csv`

- Mallet:

  `[DATASET]_[SPLIT]_[TOPICS]topics_A[ALPHA]_B[BETA]_NI[ITERATIONS]_NT[THREADS]_RND[SEED]_mallet_[METRIC].csv`

3 Setting up the Machine

Hardware requirements

- An x86-64 Linux machine with at least 24 CPU cores (all experiments use 24 threads).
- 512 GB of RAM
- 750 GB of free disk space on the file system that holds the repository (datasets, dependencies, and the model checkpoints written during the runs).
- Internet access (dependencies and datasets are downloaded by the scripts).

We ran the experiments on our lab server with 2 × Intel Xeon E5-2683 v4 CPUs (32 cores, 64 hardware threads), 512 GB of RAM, and Red Hat Enterprise Linux 8.10. Its technical properties are reported in Appendix B.

We also reproduced Table 1 on a bare-metal node of the Chameleon testbed [3] with 2 × Intel Xeon E5-2670 v3 CPUs (24 cores, 48 hardware threads) and 512 GB of RAM, where the whole pipeline took about 5 days (its technical properties are also reported in Appendix B). Reviewers who do not have access to a similar machine can use Chameleon: instructions for reserving and setting up a node are in `settingUpChameleon.md` at the root of the repository (`https://github.com/samihadouaj/SFDB-SVI_SIGMOD2026_ARI/blob/main/settingUpChameleon.md`). If needed, reviewers can also contact us to get access to our lab server.

Note on Color Coding

Throughout this document, commands are color-coded to indicate the environment in which they should be executed:

- **Blue**: commands to be run on the host machine, which hosts the Docker container.
- **Orange**: commands to be run inside the Docker container.

The sections "Required Libraries and Dependencies" and "Datasets" provide an overview of the software components used in our experiments and of the datasets, respectively.

Software setup

The commands below require `sudo` rights on the host.

1. Install Docker. On Red Hat Enterprise Linux (the system we used), Docker is provided by Podman:

```
sudo dnf install podman-docker

```

   On other Linux distributions, install Docker Engine following `https://docs.docker.com/engine/install/`.

2. Install Git and tmux (on Debian/Ubuntu: `sudo apt install git tmux`):

```
sudo dnf install git tmux

```

3. Clone the repository:

```
git clone https://github.com/samihadouaj/SFDB-SVI_SIGMOD2026_ARI.git
cd SFDB-SVI_SIGMOD2026_ARI

```

For an overview of the project's structure and of its directories, please refer to Appendix C.

4 Running the Experiments

All the commands below are run from the root of the repository. On our server (ada), the commands of steps 2–4 must be run with `sudo`.

1. The experiments run for several days. Start a tmux session, so that nothing is interrupted if the connection to the host is lost:

```
tmux

```

2. Within the tmux session, build the Docker image `starfishdb_dev_env_docker_img` (CentOS 7.9.2009 with the required toolchains and libraries):

```
./scripts/build_devenv_docker_img.sh

```

3. Create the container. The script prints the exact command to access it. The repository is mounted in the container at `/gammapdb_arrow`. The container's name follows the pattern: `starfishdb_dev_env_container_<username><hash>`

```
./scripts/create_devenv_docker_cont.sh

```

4. Access the container, replacing `<container name>` with the name printed in step 3:

```
docker exec -it <container name> bash

```

5. Run `main_script.sh`. It downloads and builds the dependencies, downloads and preprocesses the datasets, builds StarfishDB and C++SVI, runs all the experiments, and computes Table 1:

```
bash /gammapdb_arrow/scripts/main_script.sh

```

   Note: This takes about 4 days on our lab server. The script stops at the first failing step.

6. You can detach from the tmux session with `Ctrl-b d` and re-attach to it later with:

```
tmux attach

```

IMPORTANT: T1 and T2 are derived from the measured running times. Run only one experiment at a time, and avoid running other heavy workloads on the machine while the experiments are running.

5 Details of main_script.sh

This section can be skipped if the previous section finished execution without any problems.

`main_script.sh` runs the following steps, all located in the `scripts/` folder, in this order. The durations were measured on our lab server (Appendix B); they include the evaluation of all checkpoints.

Step	Script	What it does	Duration
1	`get_deps.sh`	Downloads and builds ClangJIT and Apache Arrow; installs TeX Live	∼1 h
2	`get_uci_datasets.sh`	Builds Mallet; downloads and preprocesses PubMed	∼2 h
3	`get_wiki_dataset.sh`	Downloads and preprocesses Wikipedia	∼1.5 h
4	`build_all.sh`	Builds StarfishDB and C++SVI	minutes
5	`run_pubmed_sfdb_svi.sh`	PubMed: SFDB-SVI	∼7.5 h
6	`run_pubmed_mallet_cgs.sh`	PubMed: Mallet, then SFDB-CGS	∼18 h
7	`run_pubmed_cxx_svi.sh`	PubMed: C++SVI	∼8 h
8	`run_wiki_sfdb_svi.sh`	Wikipedia: SFDB-SVI	∼12 h
9	`run_wiki_mallet_cgs.sh`	Wikipedia: Mallet, then SFDB-CGS	∼31 h
10	`run_wiki_cxx_svi.sh`	Wikipedia: C++SVI	∼14 h
11	`make_loglik_table.py`	Computes T1, T2 and Table 1	seconds
12	`../report/make_latex_table.py`	Writes and compiles `report/loglik_tables.pdf`	seconds

At the end of step 4, the log reports `StarfishDB build: SUCCEEDED` and `C++SVI build: SUCCEEDED`.

In case of a network error or of a script failure, the steps can be run one by one, inside the container, in the order of the table above. To resume after a failure, run only the failed step and the ones after it. The first three lines prepare the environment and must be run in every new shell:

```
cd /gammapdb_arrow
source /opt/rh/devtoolset-11/enable
export PATH=$PATH:/usr/local/texlive/2024/bin/x86_64-linux

bash scripts/get_deps.sh            2>&1 | tee logs/log_get_deps.txt
bash scripts/get_uci_datasets.sh    2>&1 | tee logs/log_get_uci_datasets.txt
bash scripts/get_wiki_dataset.sh    2>&1 | tee logs/log_get_wiki_dataset.txt
bash scripts/build_all.sh           2>&1 | tee logs/log_build_all.txt

bash scripts/run_pubmed_sfdb_svi.sh   2>&1 | tee logs/log_run_pubmed_sfdb_svi.txt
bash scripts/run_pubmed_mallet_cgs.sh 2>&1 | tee logs/log_run_pubmed_mallet_cgs.txt
bash scripts/run_pubmed_cxx_svi.sh    2>&1 | tee logs/log_run_pubmed_cxx_svi.txt
bash scripts/run_wiki_sfdb_svi.sh     2>&1 | tee logs/log_run_wiki_sfdb_svi.txt
bash scripts/run_wiki_mallet_cgs.sh   2>&1 | tee logs/log_run_wiki_mallet_cgs.txt
bash scripts/run_wiki_cxx_svi.sh      2>&1 | tee logs/log_run_wiki_cxx_svi.txt

python3 scripts/make_loglik_table.py 2>&1 | tee logs/log_make_loglik_table.txt
python3 report/make_latex_table.py   2>&1 | tee logs/log_make_latex_table.txt
```

IMPORTANT: Each step of `main_script.sh` saves its output to its own log file in the `logs/` folder of the repository: `logs/log_<script name>.txt`. In the unlikely case that an error occurs, the log of the failing step shows its cause. If the error persists, please contact us at shadouaj@umich.edu and attach the content of the `logs/` folder.

6 Results

At the end of `main_script.sh`:

- `make_loglik_table.py` prints two tables in the terminal (also saved to `logs/log_make_loglik_table.txt`): the reproduced Table 1, and the reproduced values next to the values reported in the paper.
- `make_latex_table.py` writes the same two tables to `report/loglik_tables.tex` and compiles them to `report/loglik_tables.pdf`. Since the repository is mounted in the container, the PDF is also available on the host, in the `report/` folder of the repository.

Both scripts can be re-run at any time from `/gammapdb_arrow`:

```
python3 scripts/make_loglik_table.py
python3 report/make_latex_table.py
```

They expect exactly one run per approach and dataset in `benchmarks/csv/`.

How Table 1 is computed

For each approach, `make_loglik_table.py` reads the test log-likelihood at each checkpoint (`*_test_*_loglik.csv`) and the cumulative training time at each checkpoint, averaged over the timing runs (`*_test_*_plotdata.csv`). Then:

- **T1** is the training time of SFDB-SVI after doing one sweep of the training data;
- **T2** is the training time of Mallet at its last checkpoint (iteration 40);
- the log-likelihood of each approach at T1 and T2 is linearly interpolated between its two surrounding checkpoints. If a run ends before T1 or T2, its last value is used.

T1 and T2 are computed from the reproduced runs.

Table 1 in the paper

Dataset	**Timestamp**	**SFDB-SVI**	SFDB-CGS	C++SVI	Mallet
PubMed (100 topics)	T1 (773 s)	**−4.0673**	−4.0767	−4.0685	−4.0736
	T2 (971 s)	**−4.0664**	−4.0733	−4.0679	−4.0690
Wikipedia (200 topics)	T1 (1659 s)	**−5.5353**	−5.6240	−5.6567	−5.7077
	T2 (3281 s)	**−5.5362**	−5.6173	−5.6099	−5.6281

Note: Log-likelihood values are scaled by 10^8. Higher is better.

What to verify

In the paper, Table 1 addresses two questions. The reproduced table should support the same answers:

1. **Can a PPDL program generate an inference method that competes with a hard-coded implementation?** At both timestamps, the log-likelihoods of the models trained with StarfishDB (SFDB-SVI and SFDB-CGS) are largely comparable to those of the specialized tools (C++SVI and Mallet).

2. **How does the new variational method compare with the Monte Carlo method of StarfishDB?** SFDB-SVI converges faster: its log-likelihood is essentially stable from T1 onward, while the Monte Carlo methods (SFDB-CGS and Mallet) keep improving between T1 and T2.

We successfully reproduced Table 1 on two different machines: our lab server (Appendix B) and a Chameleon node (Section 3). Both reproductions support the same answers to the two questions above.

Expected differences from the paper

- T1 and T2 depend on the machine, so the reproduced timestamps may differ from those in the paper, and the log-likelihoods read at those timestamps shift accordingly.
- Small variations between runs are expected.

7 Required Libraries and Dependencies

The provided scripts (the Dockerfile and `get_deps.sh`, `get_uci_datasets.sh`, `build_all.sh`) automatically handle the installation of all necessary dependencies. This section provides an overview of the main components; the complete list is in Appendix A.

1. ClangJIT (LLVM project with C++ JIT support), used to build StarfishDB
   - Version: commit 91084ef018240bbb8e24235ff5cd8c355a9c1a1e, with `patches/clangjit.patch`
   - Source: `https://github.com/hfinkel/llvm-project-cxxjit.git`
2. Apache Arrow
   - Version: 9.0.0
   - Source: `https://archive.apache.org/dist/arrow/arrow-9.0.0/apache-arrow-9.0.0.tar.gz`
3. Mallet (MAchine Learning for LanguagE Toolkit)
   - Version: v202108, with `patches/mallet.patch`
   - Source: `https://github.com/mimno/Mallet.git`
4. GCC 11 (devtoolset-11), used to build C++SVI and the preprocessing tools
5. TeX Live (basic scheme, with the `booktabs`, `multirow`, `caption` and `geometry` packages), used to typeset the tables
   - Source: CTAN mirror

The Docker container is based on CentOS 7.9.2009 and includes the development tools, Java, and Python 3 with the libraries needed by the scripts. The host machine requires Docker, Git and tmux.

8 Datasets

The experiments use the following datasets. They are downloaded and preprocessed by `get_uci_datasets.sh` and `get_wiki_dataset.sh` and stored in the `data/` folder of the repository. Each dataset is split into a training set (95% of the documents) and a test set (5%) with a fixed random seed. The log-likelihood reported in Table 1 is computed on the test set.

1. PubMed
   - Source: UCI Machine Learning Repository (bag-of-words datasets)
   - Description: a collection of PubMed abstracts
   - Statistics:
     - Documents (D): 8,200,000 (7,789,502 training, 410,498 test)
     - Vocabulary size (W): 141,043
     - Total words (N): approximately 730,000,000
2. Wikipedia

- Source: a corpus of English Wikipedia articles and a stop-word list, downloaded from the authors' Dropbox by `get_wiki_dataset.sh`
- Description: the first 4,000,000 articles of the corpus; stop words and words that appear in fewer than 50 documents are removed
- Statistics:
  - Documents (D): 4,000,000 (3,799,409 training, 200,591 test)
  - Vocabulary size (W): 192,095

`get_uci_datasets.sh` also prepares the smaller UCI datasets KOS and NYTimes; they are not used for Table 1.

Appendix A: List of Dependencies

This appendix lists all the dependencies used in our experiments.

Host Machine Requirements

- Docker (or Podman with `podman-docker`)
- Git
- Tmux

Docker Container (CentOS 7.9.2009)

- Development Tools group
- CMake3
- Ninja-build
- Which
- Screen
- Devtoolset-11
- Devtoolset-7
- Git
- Python 3
- Sudo
- CMake
- Java 11 OpenJDK
- Ant
- Nano
- Snappy
- GSL and GSL-devel
- Bzip2
- Wget
- BC (Basic Calculator)

Python packages:

- Mpmath
- NumPy

- Pandas
- hmmlearn

Installed by the scripts:

- ClangJIT (commit 91084ef018240bbb8e24235ff5cd8c355a9c1a1e)
- Apache Arrow 9.0.0
- Mallet v202108
- TeX Live (basic scheme) with the `booktabs`, `multirow`, `caption` and `geometry` packages

Appendix B: Machine Configuration

This appendix reports the technical specifications of the two machines on which we reproduced the experiments: our lab server and a node of the Chameleon testbed.

Our Lab Server

Operating System Information

```
Linux 4.18.0-553.162.1.el8_10.x86_64 #1 SMP x86_64 x86_64 x86_64 GNU/Linux

Red Hat Enterprise Linux release 8.10 (Ootpa)
```

CPU Information

```
Architecture:        x86_64
CPU op-mode(s):      32-bit, 64-bit
Byte Order:          Little Endian
CPU(s):              64
On-line CPU(s) list: 0-63
Thread(s) per core:  2
Core(s) per socket:  16
Socket(s):           2
NUMA node(s):        2
Vendor ID:           GenuineIntel
CPU family:          6
Model:               79
Model name:          Intel(R) Xeon(R) CPU E5-2683 v4 @ 2.10GHz
Stepping:            1
CPU max MHz:         3000.0000
CPU min MHz:         1200.0000
BogoMIPS:            4199.99
Virtualization:      VT-x
L1d cache:           32K
L1i cache:           32K
L2 cache:            256K
L3 cache:            40960K
NUMA node0 CPU(s):   0,2,4,...,62 (even CPUs)
NUMA node1 CPU(s):   1,3,5,...,63 (odd CPUs)
```

Memory Information

```
              total
Mem:          503Gi
Swap:         4.0Gi
```

Storage

```
Size of the file system holding the repository: 1.0T
```

Software

```
Podman 4.9.4 (Docker CLI via podman-docker)
tmux 2.7
git 2.43.7
```

Chameleon Node (CHI@TACC)

This is the bare-metal node we reserved on the CHI@TACC site of the Chameleon testbed [3] and on which we reproduced the experiments (see `settingUpChameleon.md`).

Operating System Information

```
Linux 5.15.0-185-generic x86_64 GNU/Linux

Ubuntu 22.04.5 LTS
```

CPU Information

```
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                48
On-line CPU(s) list:   0-47
Thread(s) per core:    2
Core(s) per socket:    12
Socket(s):             2
NUMA node(s):          2
Vendor ID:             GenuineIntel
CPU family:            6
Model:                 63
Model name:            Intel(R) Xeon(R) CPU E5-2670 v3 @ 2.30GHz
Stepping:              2
CPU max MHz:           3100.0000
CPU min MHz:           1200.0000
BogoMIPS:              4600.06
Virtualization:        VT-x
L1d cache:             768 KiB (24 instances)
L1i cache:             768 KiB (24 instances)
L2 cache:              6 MiB (24 instances)
L3 cache:              60 MiB (2 instances)
NUMA node0 CPU(s):     0,2,4,...,46 (even CPUs)
NUMA node1 CPU(s):     1,3,5,...,47 (odd CPUs)
```

Memory Information

```
               total
Mem:           503Gi
Swap:          0B
```

Storage

Size of the file system holding the repository: 1.8T (NVMe SSD)

Software

Docker 29.9.2
tmux 3.2a
git 2.34.1

Appendix C: Project Structure

This appendix provides an overview of the project structure, detailing the main components and their purposes.

Root Directory

- `CMakeLists.txt`: main CMake configuration file of StarfishDB
- `README.md`: project readme file, with a summary of this guide
- `settingUpChameleon.md`: instructions for reserving and setting up a Chameleon node
- License files (`LICENSE`, `NOTICE.txt`)

Source Directories

- `src/`: StarfishDB source code, including the SVI engine (main application file: `gammapdb_arrow.cpp`)
- `cxx_svi/`: C++SVI source code
- `extras/`: dataset preprocessing tools; Mallet is installed in `extras/mallet/`
- `scripts/`: setup, build and experiment scripts; `scripts/dockerfiles/` contains the Dockerfile
- `report/`: LaTeX table generator (`make_latex_table.py`) and its output
- `conf/`: configuration files (UCI dataset URLs, TeX Live installation profile)
- `patches/`: patches applied to ClangJIT and Mallet

Directories Created by the Scripts

- `external/`: sources of ClangJIT and Apache Arrow
- `libs/`: compiled ClangJIT and Apache Arrow
- `build/`: StarfishDB build (`build/gammapdb_arrow`)
- `data/`: raw and preprocessed datasets
- `benchmarks/`: results of the experiments (`benchmarks/csv/`) and temporary model checkpoints
- `logs/`: log of every step

Key Files in scripts/

- `main_script.sh`: runs the entire pipeline
- `build_devenv_docker_img.sh`, `create_devenv_docker_cont.sh`: build the Docker image and create the container
- `get_deps.sh`, `get_uci_datasets.sh`, `get_wiki_dataset.sh`: download and build the dependencies and the datasets
- `build_all.sh`: builds StarfishDB and C++SVI

- `run_{pubmed,wiki}_{sfdb_svi,mallet_cgs,cxx_svi}.sh`: run the experiments
- `make_loglik_table.py`: computes Table 1

Acknowledgments

We used the Chameleon testbed [3] for one of our two reproductions of the experiments.

References

[1] Ouael Ben Amara, Sami Hadouaj, and Niccolò Meneghetti. Starfishdb: A query execution engine for relational probabilistic programming. *Proc. ACM Manag. Data*, 2(3):185, 2024.

[2] Sami Hadouaj, Ouael Ben Amara, and Niccolò Meneghetti. Variational inference for de finetti logic. *Proc. ACM Manag. Data*, 4(3), 2026.

[3] Kate Keahey, Jason Anderson, Zhuo Zhen, Pierre Riteau, Paul Ruth, Dan Stanzione, Mert Cevik, Jacob Colleran, Haryadi S. Gunawi, Cody Hammock, Joe Mambretti, Alexander Barnes, François Halbach, Alex Rocha, and Joe Stubbs. Lessons learned from the Chameleon testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, July 2020.

[4] Andrew Kachites McCallum. MALLET: A machine learning for language toolkit. `http://mallet.cs.umass.edu`, 2002.